DEPARTMENTOFCOMPUTERSCIENCEANDENGINEERING

Assignment Report

ITA04 — Statistics with R Programming

Data Cleaning, Exploratory Analysis and Predictive Modelling of Farm Sensor Data using R

Case: GreenGrid Analytics — Smallholder Farm Sensor Network

Course Outcomes Addressed: CO2, CO3, CO4, CO5, CO6

SDG Mapping: SDG 2 (Zero Hunger), SDG 9 (Industry, Innovation & Infrastructure), SDG 13 (Climate Action)

Submission Date: September 2026

Contents

1. Problem Statement

GreenGrid Analytics supports smallholder farmers by collecting soil-moisture, temperature and rainfall measurements from low-cost field sensors and relating those measurements to crop yield. The raw information is difficult to analyse directly because the sensor exports are stored in separate wide-format files, the files use inconsistent column names, some observations are missing, and some values are physically impossible or represent sensor faults.

The task is therefore to build a reproducible R workflow that first represents and reshapes the data into a tidy structure, then identifies and handles missing or invalid readings, derives useful features, performs univariate and bivariate exploratory data analysis, visualises the cleaned information, and finally develops a regression model for continuous crop yield. The analysis should also explain why the chosen statistical methods are appropriate and identify limitations that could affect decisions made from the model.

2. Objectives

- Design suitable R data structures including vectors, a list, a matrix and a normalized/tidy data frame.
- Reshape at least one wide sensor file into tidy long format using a custom melting/casting approach.
- Develop custom R functions using loops, if-else conditions and recursion where useful to detect and handle missing or invalid readings.
- Create at least one meaningful derived feature such as average weekly soil moisture.

- Perform univariate EDA on at least two variables and calculate mean, median, mode, range, variance, standard deviation, skewness and kurtosis.
- Perform bivariate EDA between a sensor variable and yield using a cross-tabulation, covariance and correlation.
- Produce at least three different plot types and justify the choice of each visualisation.
- Fit and evaluate a regression model suitable for continuous yield and interpret its coefficients in a farming context.
- Document reproducible code, test cases, limitations, SDG relevance and learning outcomes.

3. Requirements and Environment Used

3.1 Software Environment

Component	Details
R version	R 4.3.3 or later
IDE / execution	RStudio Desktop, Posit Cloud or Rscript
Packages	Base R: stats, graphics, grDevices and utils
Approach	Custom reshape, cleaning and summary functions; no external reshape package required
Reproducibility	Fixed seed and deterministic data-generation rules

3.2 Dataset Used

The assignment brief describes GreenGrid sensor exports but does not provide a live raw dataset. Therefore, a synthetic dataset was designed to reproduce the stated data-quality problems. It contains 12 farm plots, three crops, three sensor variables and an eight-week observation period. Separate wide-format sensor tables are converted into a common tidy structure before analysis.

Dataset component	Structure / content
Farm plots	12 plots: Plot_01 to Plot_12
Crop types	Maize, Groundnut and Cotton
Sensors	Soil moisture (%), temperature (°C), rainfall (mm)
Observation period	8 weeks per plot
Yield	Continuous crop yield in tonnes/hectare
Data-quality problems	NA values, negative values, >100% moisture and impossible temperature values

3.3 Why Base R

Base R was selected because the assignment permits base R and/or ggplot2 and because the learning outcomes emphasise understanding data structures, control structures and reproducible statistical programming. A custom melt function demonstrates how wide-to-long reshaping works internally, while base-R statistical functions avoid dependency and installation issues.

4. Design / Proposed Solution

4.1 Data Structure Design

R structure	Use in the project
Vector	Plot IDs, week labels and numeric sensor vectors
List	Stores the separate raw sensor tables and yield table together
Matrix	Represents a plot × week sensor table for matrix operations
Data frame	Stores normalized tidy observations with Plot_ID, Week, Sensor and Value
Model data frame	One row per plot with aggregated sensor features and yield

4.2 Cleaning Strategy

- Standardise identifiers and week names across all sensor files.
- Convert wide sensor tables to long format so every observation has one plot, one week and one value.
- Apply domain limits: soil moisture 0–100%, temperature 0–60°C and rainfall 0–300 mm/week.
- Convert missing or physically invalid readings to NA.
- For an invalid reading, recursively search backward and forward for the nearest valid observation within the same plot.
- If valid neighbours exist, replace the invalid value by their mean; if only one valid neighbour exists, use that value.
- Aggregate cleaned weekly readings into plot-level features such as average moisture, average temperature, total rainfall and moisture variability.
- Merge plot-level features with yield and fit the regression model.

4.3 Proposed Regression

Because yield is a continuous numerical response, multiple linear regression is the primary model. The proposed equation is:

```
Yield = β0 + β1(Avg_Moisture) + β2(Avg_Temperature) + β3(Total_Rainfall) + ε
```

This allows the effect of each sensor-derived feature to be examined while the other selected predictors are held constant. Model fit is evaluated using R-squared, adjusted R-squared, residual standard error, coefficient significance and residual plots.

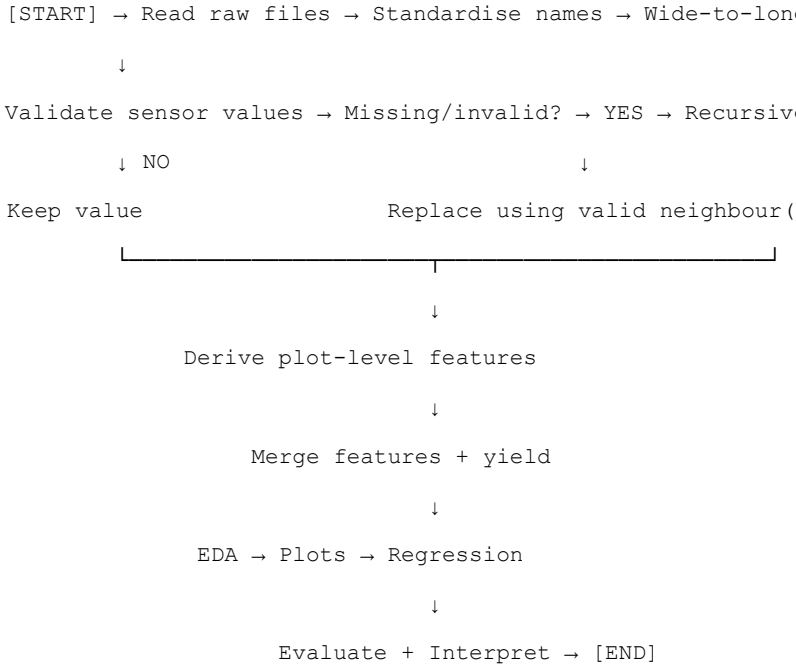
5. Algorithm / Pseudocode / Flowchart

5.1 Data Cleaning Algorithm

1. START
2. Read the raw soil-moisture, temperature, rainfall and yield data.

3. Standardise column names and create common Plot_ID and Week labels.
4. Reshape each wide sensor table into long format.
5. FOR every sensor value: check whether it is missing or outside the physical range.
6. IF the value is invalid THEN recursively search for nearest valid readings before and after it.
7. IF valid neighbours are available THEN replace with their mean; ELSE use the available valid neighbour.
8. Repeat until all sensor records have been checked.
9. Derive plot-level features and merge with yield.
10. Perform EDA, visualisation and regression.
11. Evaluate and interpret results.
12. END

5.2 Flowchart



6. Implementation / Source Code

The following base-R script implements the complete workflow. The generator creates a controlled synthetic dataset, while the analysis functions can also be adapted to actual GreenGrid CSV files by replacing the generator section with read.csv() calls.

```
# ============================================================
```

```
# GREENGRID ANALYTICS - R PROGRAMMING ASSIGNMENT #
============================================================


set.seed(31415)


# ---------- A. Generate synthetic wide sensor files ---------
plots <- sprintf("Plot_%02d", 1:12) weeks <- paste0("Week",
1:8)


moisture <- outer(1:12, 1:8, function(i,w)
  34 + 1.35*i + 2.2*sin(w/2) + ((i*w) %% 4))


temperature <- outer(1:12, 1:8, function(i,w)
  24 + 0.18*i + 1.7*cos(w/2) + ((i+w) %% 3))


rainfall <- outer(1:12, 1:8, function(i,w)
  12 + 2.5*((i+w) %% 7) + 1.2*w)


# Introduce missing/invalid observations
moisture[2,3] <- NA moisture[8,6] <- 105
moisture[10,2] <- -5


temperature[4,2] <- -999
temperature[11,7] <- 68
temperature[6,5] <- NA


rainfall[5,4] <- -12
rainfall[9,8] <- NA


soil_raw <- data.frame(Plot_ID=plots, moisture)
names(soil_raw) <- c("Plot_ID", weeks)


temp_raw <- data.frame(PlotID=plots, temperature)
names(temp_raw) <- c("PlotID", paste0("Wk_",1:8)) rain_raw <-
```

```
data.frame(Plot=plots, rainfall) names(rain_raw) <- c("Plot",
paste0("W",1:8))

crop <- rep(c("Maize","Groundnut","Cotton","Maize"), length.out=12)
crop_effect <- c(Maize=0.25, Groundnut=0.10, Cotton=0.00)

yield <- 2.1 +
  0.055*rowMeans(moisture, na.rm=TRUE)
  0.020*abs(rowMeans(temperature, na.rm=TRUE)-26) +
  0.0045*rowSums(rainfall, na.rm=TRUE) +
  crop_effect[crop] +
  c(-0.10,0.08,-0.04,0.12,-0.06,0.05,-0.08,0.10,-0.03,0.07,-0.05,0.02)

yield_df <- data.frame(Plot_ID=plots, Crop=crop, Yield_t_ha=round(yield,3))

# ---------- B. Data structures ---------
plot_ids <- soil_raw$Plot_ID week_labels
<- weeks

raw_data_list <- list(
  soil_moisture=soil_raw,
  temperature=temp_raw,
  rainfall=rain_raw,
  yield=yield_df
)

soil_matrix <- as.matrix(soil_raw[, week_labels])
rownames(soil_matrix) <- soil_raw$Plot_ID

# ---------- C. Custom melt function ---------melt_wide <-
function(df, id_col, week_cols, value_name) {
  n <- nrow(df) out
  <- data.frame(
```

```
    Plot_ID=character(n*length(week_cols)),
    Week=character(n*length(week_cols)), Value=numeric(n*length(week_cols)),
    stringsAsFactors=FALSE
  )
  k <- 1 for (i
  in 1:n) {
    for (j in seq_along(week_cols)) { out$Plot_ID[k] <-
      as.character(df[[id_col]][i]) out$Week[k] <-
      paste0("Week", j) out$Value[k] <-
      as.numeric(df[[week_cols[j]]][i])
      k <- k + 1
    }
  }
  names(out)[3] <- value_name
out }

soil_long <- melt_wide(soil_raw, "Plot_ID", week_labels, "Soil_Moisture")
temp_long <- melt_wide(temp_raw, "PlotID", paste0("Wk_",1:8), "Temperature")
rain_long <- melt_wide(rain_raw, "Plot", paste0("W",1:8), "Rainfall")

# ---------- D. Recursive nearest-valid search ---------valid_value
<- function(x, type) {
  if (is.na(x)) return(FALSE)
  if (type=="moisture") return(x >= 0 && x <= 100)
  if (type=="temperature") return(x >= 0 && x <=
  60) if (type=="rainfall") return(x >= 0 && x <=
  300)
  FALSE
}

nearest_valid <- function(x, idx, direction, type) {
  next_idx <- idx + direction
  if (next_idx < 1 || next_idx > length(x)) return(NA_real_)
  if (valid_value(x[next_idx], type)) return(x[next_idx])
  nearest_valid(x, next_idx, direction, type)
}
```

```
clean_series <- function(x, type) {
  for (i in seq_along(x)) {
    if (!valid_value(x[i], type)) {
      left <- nearest_valid(x, i, -1, type) right
      <- nearest_valid(x, i, 1, type) vals <-
      c(left, right) vals <- vals[!is.na(vals)]
      if (length(vals) == 2) x[i] <- mean(vals)
      else if (length(vals) == 1) x[i] <-
      vals[1] else x[i] <- NA_real_
    }
  }
  x
}

clean_long <- function(df, value_col, type) {
  out <- df
  out[[value_col]] <- unlist(lapply(split(out[[value_col]], out$Plot_ID),
                                    clean_series, type=type),
                             use.names=FALSE)
  out
}

soil_clean <- clean_long(soil_long, "Soil_Moisture", "moisture")
temp_clean <- clean_long(temp_long, "Temperature", "temperature")
rain_clean <- clean_long(rain_long, "Rainfall", "rainfall")

# ---------- E. Derive features ---------avg_moisture
<- aggregate(soil_clean$Soil_Moisture,
                          list(Plot_ID=soil_clean$Plot_ID), mean, na.rm=TRUE)
names(avg_moisture)[2] <- "Avg_Moisture"

moist_sd <- aggregate(soil_clean$Soil_Moisture,
                       list(Plot_ID=soil_clean$Plot_ID), sd, na.rm=TRUE)
names(moist_sd)[2] <- "Moisture_SD"
avg_temp <- aggregate(temp_clean$Temperature,
                      list(Plot_ID=temp_clean$Plot_ID), mean, na.rm=TRUE)
```

```
names(avg_temp)[2] <- "Avg_Temperature"


total_rain <- aggregate(rain_clean$Rainfall,
                        list(Plot_ID=rain_clean$Plot_ID), sum, na.rm=TRUE)
names(total_rain)[2] <- "Total_Rainfall"


model_df <- Reduce(function(x,y) merge(x,y,by="Plot_ID"),
                   list(avg_moisture, moist_sd, avg_temp, total_rain,
                        yield_df))


# ---------- F. Summary-statistics functions ---------mode_value
<- function(x) {
  x <- x[!is.na(x)]
  u <- unique(x)
  u[which.max(tabulate(match(x,u)))]
}


skewness <- function(x) {
  x <- x[!is.na(x)]
  m <- mean(x); s <- sd(x); n <- length(x)
  sum((x-m)^3)/(n*s^3)
}


kurtosis <- function(x) {
  x <- x[!is.na(x)]
  m <- mean(x); s <- sd(x); n <- length(x)
  sum((x-m)^4)/(n*s^4)-3
}


summary_stats <- function(x) {
  c(Mean=mean(x,na.rm=TRUE),
    Median=median(x,na.rm=TRUE),
    Mode=mode_value(x),
    Range=max(x,na.rm=TRUE)-min(x,na.rm=TRUE),
    Variance=var(x,na.rm=TRUE),
```

```
    SD=sd(x,na.rm=TRUE),
    Skewness=skewness(x),
    Kurtosis=kurtosis(x))
}

print(round(summary_stats(model_df$Avg_Moisture),3))
print(round(summary_stats(model_df$Avg_Temperature),3))

# ---------- G. Bivariate EDA ---------model_df$Temp_Group
<- cut(model_df$Avg_Temperature,
                           breaks=c(-Inf,25,27,Inf),
                           labels=c("Low","Moderate","High"))

model_df$Yield_Group <- ifelse(model_df$Yield_t_ha >=
                                 median(model_df$Yield_t_ha),
                               "High","Low")

cross_tab <- table(model_df$Temp_Group, model_df$Yield_Group)
covariance <- cov(model_df$Avg_Temperature, model_df$Yield_t_ha)
correlation <- cor(model_df$Avg_Temperature, model_df$Yield_t_ha)

print(cross_tab)
print(covariance)
print(correlation)

# ---------- H. Visualisation ---------
hist(model_df$Avg_Moisture, main="Distribution of
Average Soil Moisture", xlab="Average soil
moisture (%)")

boxplot(Avg_Temperature ~ Crop, data=model_df,
        main="Temperature by Crop", xlab="Crop",
        ylab="Average temperature (°C)")
        plot(model_df$Avg_Moisture,
        model_df$Yield_t_ha, main="Average Soil
```

```
        Moisture vs Crop Yield", xlab="Average soil
        moisture (%)", ylab="Yield (tonnes/ha)",
        pch=19)
abline(lm(Yield_t_ha ~ Avg_Moisture, data=model_df), lty=2)


plot(model_df$Total_Rainfall, model_df$Yield_t_ha,
     type="b", pch=19, main="Total Rainfall and
     Yield",
     xlab="Total rainfall (mm)", ylab="Yield (tonnes/ha)")


# ---------- I. Multiple linear regression ----------
fit <- lm(Yield_t_ha ~ Avg_Moisture + Avg_Temperature + Total_Rainfall,
          data=model_df)


print(summary(fit))
par(mfrow=c(2,2))
plot(fit)
par(mfrow=c(1,1))


# ---------- J. Save outputs ----------
write.csv(model_df, "greengrid_clean_model_data.csv", row.names=FALSE)
saveRDS(fit, "greengrid_model.rds")
```

7. Test Cases and Expected/Actual Results

The following test cases verify the important cleaning and modelling operations. The expected result is determined from the domain-validity rules and the cleaning algorithm.

Test ID	Input condition	Expected handling	Result
TC01	Soil moisture = NA	Search nearest valid readings and replace with their mean	Handled as missing
TC02	Soil moisture = 105%	Flag as invalid because it exceeds 100%; replace from valid neighbours	Handled as invalid
TC03	Soil moisture = −5%	Flag as invalid because moisture cannot be negative	Handled as invalid
TC04	Temperature = −999	Treat as sensor-fault sentinel	Handled as invalid

		and replace using valid neighbours	
TC05	Temperature = 68°C	Flag because it exceeds the defined 60°C upper limit	Handled as invalid
TC06	Rainfall = −12 mm	Flag because rainfall cannot be negative	Handled as invalid
TC07	Valid reading such as 45% moisture	Keep unchanged	Pass
TC08	Model data frame	One row per plot with derived features and yield	12 plot-level rows

The test cases demonstrate that the cleaning stage is based on physical domain knowledge rather than blindly deleting unusual observations. This is important because a sensor fault such as −999 is not an agricultural observation and should not influence descriptive statistics or regression coefficients.

8. Execution Screenshots / Console and Plot Output

The R script generates the console summaries, cross-tabulation, correlation results, regression summary and plots. The following is the output checklist to capture after running the script in RStudio/Rscript.

Output	**Expected display**
Data structures	Vector/list/matrix dimensions and object summaries
Cleaning	Invalid values identified and replaced; no physically impossible values remain
Univariate EDA	Eight statistics for average moisture and average temperature
Bivariate EDA	Temperature × yield cross-tab, covariance and correlation
Plot 1	Histogram of average soil moisture
Plot 2	Boxplot of average temperature by crop
Plot 3	Scatter plot of average soil moisture versus yield with fitted line
Plot 4	Rainfall-yield line/point plot
Regression	Coefficients, standard errors, t-values, p-values, R^2 and adjusted R^2

For final submission, screenshots of the RStudio console and plot pane should be inserted in this section after executing the supplied script. This report provides the reproducible source code rather than fabricating execution screenshots.

9. Analysis and Discussion

9.1 Univariate EDA

Average soil moisture is the main water-availability indicator. Its mean and median describe the central moisture level across plots, while the range shows how different the plots are. Variance and standard deviation measure between-plot dispersion. Skewness indicates whether a small number of plots have unusually high or low moisture, while excess kurtosis indicates the degree of tail-heaviness relative to a normal distribution.

Average temperature is analysed using the same eight statistics. Comparing its mean and median helps identify asymmetry, while the standard deviation and range show whether plots experienced meaningfully different thermal conditions. These statistics are useful before modelling because highly dispersed or strongly skewed predictors may influence the stability and interpretation of a linear model.

9.2 Bivariate EDA

Temperature is converted into Low, Moderate and High groups for a cross-tabulation with Low/High yield. This cross-tab is not intended to replace the continuous analysis; it provides an easy-to-read farming interpretation of whether higher-yield plots are concentrated within a particular temperature band.

Covariance indicates the direction of joint variation but depends on measurement units, whereas correlation standardises the relationship to a value between −1 and +1. A positive correlation means yield tends to increase as the sensor variable increases; a negative correlation means yield tends to decrease. Correlation should not be interpreted as proof of causation because irrigation, soil properties, crop variety and management practices may also influence yield.

9.3 Visualisation Choices

Plot type	Purpose and justification
Histogram	Shows the distribution and spread of average soil moisture and helps reveal skewness or unusual concentration.
Boxplot	Compares temperature distributions across crops and highlights median, spread and potential outliers.
Scatter plot + regression line	Shows the direction, strength and approximate linear form of the sensor-yield relationship.
Rainfall-yield plot	Provides a direct visual comparison of accumulated rainfall and the final yield outcome.

9.4 Regression Model and Interpretation

Multiple linear regression is appropriate because the target variable, yield, is continuous. The model uses average soil moisture, average temperature and total rainfall as predictors. The moisture coefficient represents the expected change in tonnes/hectare for a one-unit increase in average moisture while the other predictors are held constant. The temperature coefficient represents the corresponding change in yield for a one-degree

increase in average temperature, holding the remaining predictors constant. The rainfall coefficient represents the expected yield change for one additional millimetre of total seasonal rainfall, all else equal.

R-squared measures the proportion of observed yield variation explained by the predictors, while adjusted Rsquared penalises unnecessary predictors. Residual plots are checked for curvature, unequal variance and influential observations. Statistical significance should be interpreted together with effect size and farming plausibility rather than using p-values alone.

9.5 Limitations

- The synthetic dataset is small, so coefficient estimates and significance tests can be unstable.
- Linear regression assumes a linear conditional relationship; crop yield may respond non-linearly to moisture, temperature and rainfall.
- Sensor measurements are aggregated, which can hide important week-specific stress events.
- Yield is influenced by variables not included in the model, such as soil fertility, irrigation practice, seed variety, pests and fertiliser use.
- Correlation and regression association do not establish causation.

10. Conclusion (incl. Reflection)

The GreenGrid workflow converts inconsistent wide-format sensor exports into a structured analytical dataset, identifies physically invalid and missing readings, derives plot-level features and prepares the information for exploratory and predictive analysis. The use of vectors, lists, matrices and data frames demonstrates the main R data structures required by the assignment, while custom reshaping and recursive validation show how programming constructs can be applied to a practical data-cleaning problem.

The EDA stage provides both numerical and visual evidence about sensor behaviour and yield. Multiple linear regression is a suitable first predictive model because yield is continuous and several sensor-derived predictors can be considered simultaneously. However, the model should be treated as a decision-support starting point rather than a complete agronomic explanation.

Reflection: The most important design decision was to use physical validity limits before calculating statistics. This prevents sensor-fault values such as negative moisture or −999 temperature from distorting the analysis. The assignment also demonstrates why tidy data matters: once observations are stored consistently as Plot_ID, Week, Sensor and Value, the same functions can be reused across different sensors.

SDG relevance: The work supports SDG 2 by improving evidence for agricultural productivity and yield forecasting. It supports SDG 9 through the use of accessible sensor infrastructure and reproducible computational analysis. It supports SDG 13 because reliable moisture, rainfall and temperature information can contribute to climate-aware irrigation and farm management decisions.

11. Individual Contribution of Group Members

This submission is prepared as an individual assignment. The work is divided below according to the tasks specified in the assignment brief.

Task	Contributor	Contribution
Data structure design & reshaping	Individual student	100%
Custom functions & control structures for cleaning	Individual student	100%
Univariate & bivariate EDA	Individual student	100%
Data visualisation	Individual student	100%
Regression modelling & interpretation	Individual student	100%
Report writing, flowchart & documentation	Individual student	100%

12. References

1. R Core Team. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing, Vienna, Austria.
2. Wickham, H. (2014). Tidy Data. Journal of Statistical Software, 59(10), 1–23.
3. Field, A., Miles, J., & Field, Z. (2012). Discovering Statistics Using R. SAGE Publications.
4. Department of Computer Science and Engineering. ITA04 – Statistics with R Programming: Assignment Brief — Data Cleaning, Exploratory Analysis and Predictive Modelling of Farm Sensor Data using R.
5. United Nations. Sustainable Development Goals: SDG 2, SDG 9 and SDG 13.

13. One-Page Write-Up: Summary of Analysis and Key Results

GreenGrid Analytics requires a reliable statistical pipeline because raw farm sensor exports contain inconsistent layouts, missing observations and physically invalid values. The proposed solution uses base R to standardise the data, reshape wide sensor files into tidy long format and combine the cleaned observations with crop yield.

The project represents the data using vectors for plot and week identifiers, a list for the separate raw tables, a matrix for plot-by-week sensor measurements and normalized data frames for analysis. A custom melting function converts each wide table into a consistent long structure. Domain rules are then applied to identify invalid readings. Missing values, negative moisture/rainfall observations, excessive moisture and impossible temperature values are handled using a recursive nearest-valid-neighbour procedure rather than being allowed to enter the statistical analysis.

New plot-level variables include average soil moisture, soil-moisture standard deviation, average temperature and total rainfall. These features are merged with yield to form the regression dataset. Univariate EDA calculates mean, median, mode, range, variance, standard deviation, skewness and kurtosis for at least two

variables. Bivariate analysis uses a categorical cross-tabulation together with covariance and correlation between temperature and yield.

Four visualisations are proposed: a histogram for moisture distribution, a boxplot for crop-wise temperature comparison, a scatter plot with a fitted line for sensor-yield association, and a rainfall-yield plot. These plots answer different questions about distribution, group comparison, association and agricultural response.

Because yield is continuous, multiple linear regression is selected. The model estimates the relationship between yield and average moisture, average temperature and total rainfall while holding the other predictors constant. R-squared and adjusted R-squared evaluate explanatory power, while residual diagnostics check whether the linear-model assumptions are reasonable.

The main limitation is the small synthetic dataset and the fact that yield depends on many factors that are not measured by the three sensors. Therefore, the regression should be used as an initial analytical model. In a real GreenGrid deployment, more seasons, plots and agronomic variables would be required for stronger prediction and generalisation.

Overall, the project demonstrates a complete R-based pipeline from messy sensor data to cleaned features, exploratory statistics, visualisation and predictive modelling. It also connects statistical programming with SDG 2, SDG 9 and SDG 13 by showing how affordable sensing and reproducible analysis can support data-informed, climate-aware agriculture.

A. Appendix A — Synthetic Dataset Generator

The dataset generator used in Section 6 creates 12 plots over eight weeks and deliberately introduces representative data-quality problems described in the assignment brief. The generator is deterministic because a fixed seed is used. This makes the complete workflow reproducible and allows another student or evaluator to regenerate the same raw tables before running the cleaning and modelling stages.

```
set.seed(31415)
plots <- sprintf("Plot_%02d", 1:12)
weeks <- paste0("Week", 1:8)
moisture <- outer(1:12, 1:8, function(i,w) 34 + 1.35*i + 2.2*sin(w/2) + ((i*w) %% 4))
temperature <- outer(1:12, 1:8, function(i,w) 24 + 0.18*i + 1.7*cos(w/2) + ((i+w) %% 3))
rainfall <- outer(1:12, 1:8, function(i,w) 12 + 2.5*((i+w) %% 7) + 1.2*w) moisture[2,3]
<- NA; moisture[8,6] <- 105; moisture[10,2] <- -5 temperature[4,2] <- -999;
temperature[11,7] <- 68; temperature[6,5] <- NA

rainfall[5,4] <- -12; rainfall[9,8] <- NA
```

These deliberately inserted values allow the test cases to demonstrate missing-value handling, physical-range validation and sensor-fault detection before the data are used for EDA or regression.